# Student Setup Guide — Agentic AI Workshop

Before Day 1 on 14 October you need a Google account, one Colab link, one cell run and an API key from any LLM provider. A free Neo4j Aura account (or the no-Neo4j starter instead) completes the setup at the start of the lab.

## Before Day 1: the 10-minute check

You run this once, on any laptop with a browser and a Google account. There is nothing to install.

1. Open the workshop Colab notebook: **[Colab link — add before sending]**.
2. Click **Runtime → Run all**, or run the first cell with Shift+Enter.
3. Wait about two minutes the first time. Colab downloads a small language model of roughly 90 MB.
4. The last line of output must read `Setup OK`.

The first cell installs the same packages the Day 1 notebooks use:

```
%pip install -q sentence-transformers faiss-cpu hnswlib

from sentence_transformers import SentenceTransformer
import faiss, hnswlib

model = SentenceTransformer('all-MiniLM-L6-v2')
vec = model.encode(["Setup check"])
print("Embedding shape:", vec.shape)
print("Setup OK")
```

It should print `Embedding shape: (1, 384)` and then `Setup OK`. If it does, you are done. If not, jump to *If something breaks* below.

Optional, 5 minutes: read the one-page primer, the primer, on what RAG is and why it sometimes states things that are not true. You will see this on screen in the first 15 minutes of Day 1, so reading it only helps you spot it sooner.

## The hands-on lab and what you build

In the lab you open one pre-built Colab notebook, the starter for your dataset, and run it. It already retrieves records, builds a graph (in Neo4j, or in ready-made lookup tables if you cannot use it) and answers questions. It recognises one kind of identifier, the first ID of your dataset (for example the Order ID in ORD). Your job is to teach it the rest, and to explain why.

| When | What you do |
|---|---|
| Day 1 lab | Plug in your LLM key, connect the graph store, run the starter. Add one pattern as a guided step and watch a question change from "no recorded link" to "connected". |
| Build window | Decide which other strings identify a single event, add them, run the public question pack, explain every result. |
| Day 2 | Show your patterns and your results table. The judges run a hidden question pack against your graph. |

## Choose your starter

There are five starters. They work the same way and are equally hard; only the subject, the records and the questions differ. Your team picks one at team formation on Day 1 (about five teams per dataset, so put your choice on the sign-up sheet). Use only your own dataset's file, and do not swap records or questions between starters.

| Code | Dataset | Starter (Neo4j) | Starter (no Neo4j) |
|---|---|---|---|
| ORD | Order / inventory / delivery | `starter_ORD.ipynb` | `starter_ORD_no_neo4j.ipynb` |
| HR | Employee / contractor / manager | `starter_HR.ipynb` | `starter_HR_no_neo4j.ipynb` |
| RISK | Risk / incident / problem | `starter_RISK.ipynb` | `starter_RISK_no_neo4j.ipynb` |
| PAY | Payment / shipment / invoice | `starter_PAY.ipynb` | `starter_PAY_no_neo4j.ipynb` |
| CLN | Patient / prescription / clinician | `starter_CLN.ipynb` | `starter_CLN_no_neo4j.ipynb` |

Every starter prints its dataset code (`DATASET_ID`) in the dataset cell. Leave it as it is: the judges use it to find your questions.

## Bring your own LLM key

No key is provided. Use a key from **any provider you like**; several offer free keys, so check your provider's site for current limits. A full run of the starter makes about 25 model calls.

1. Create an API key with your provider. Keep it private, and never paste it into a notebook cell you will submit.
2. In Colab, click the key icon in the left sidebar, choose **Add new secret**, name it `LLM_API_KEY`, paste the key and turn on **Notebook access**.
3. In the starter's **Your LLM provider** cell, set `LLM_MODEL` to a model your key can use. If your provider is not OpenAI but offers an OpenAI-compatible endpoint, also set `LLM_BASE_URL` to its URL (your provider's documentation lists both).
4. Run the cell. It should print a one-word reply.

**This is part of your work.** The whole notebook calls the model through one function, `call_llm(prompt)`. If your provider needs its own library, rewrite only the body of that function. It must take a prompt string, return the reply as a plain string, and use `temperature=0` so repeated runs give the same answer. The judges score your pattern registry, not which provider you picked.

## Create a Neo4j Aura account (free)

One person per team does this, during the break before the Day 1 lab. It is free and takes about 10 minutes. The starter builds its graph in this database. If you cannot use Neo4j, skip to **No Neo4j access? Use the no-Neo4j starter** below.

1. Go to https://neo4j.com/cloud/aura-free/ and sign up (signing in with Google works).
2. Choose **AuraDB Free** and create an instance. Name it anything, for example `workshop`. Wait one to two minutes until the instance shows **Running**.
3. When the credentials dialog appears, **download the credentials file straight away**. The password is shown once and cannot be recovered.
4. Open the downloaded `.txt` file. You need three values: the connection URI (it starts with `neo4j+s://`), the username (usually `neo4j`) and the password.
5. In Colab, open the key icon in the left sidebar and add three secrets with exactly these names: `NEO4J_URI`, `NEO4J_USER` (the file calls it `NEO4J_USERNAME`) and `NEO4J_PASSWORD`. Turn on **Notebook access** for each.
6. Run the starter's Neo4j connection cell. It should print `Connection successful`.
7. Show a mentor that line. Teammates who want to run the notebook themselves add the same three secrets in their own Colab; share the values privately, not in a chat group.

Two things to know. Aura Free pauses after a few days of inactivity, so open the Instances page the morning of Day 2 and make sure it says **Running**. And if a long-open notebook starts raising `ServiceUnavailable`, close the driver and create it again; the instance is fine, the old connection has gone stale.

## No Neo4j access? Use the no-Neo4j starter

If Aura sign-up is blocked on your network, or you would rather not use it, open the `_no_neo4j` version of your dataset's starter (see the table above) instead of the standard one. It keeps the graph in plain Python lookup tables that are already built for you, so there is nothing to sign up for or connect, and you skip the Neo4j secrets and the connection cell.

The task is exactly the same: extend the pattern registry, run the public question pack, explain the outcomes. The judges score your registry, so you are not at a disadvantage. You still bring your own LLM key (see above).

## Between Day 1 and Day 2: the build window

The notebook already works end to end. You do not write the pipeline; you decide what it should recognise.

1. Open the starter and run it top to bottom. It builds a graph from a short list of patterns, the **pattern registry**. The list has one entry: the first ID of your dataset (the Order ID in ORD, the Employee ID in HR, the Incident ID in RISK, the Invoice ID in PAY, the Patient ID in CLN).
2. Read the records that contain digits (one cell prints them). For each string ask: if I saw this in a different record, would it still point to the same event?
3. Add a pattern for every string that does. Each entry needs an example from the data and a reason. These are **Kind 1** identifiers: they name one person, case, order or document.
4. Some strings repeat across unrelated events: a place, a level or block, an amount or count, a period of time. These are **Kind 2**. Do not link on them. List the ones you considered under `REJECTED_CANDIDATES`, with the reason.
5. Run the **public question pack**: 8 questions where the right answer is "no recorded link" and 3 where it is "yes". Fill in `OUTCOMES` (see *Filling in the outcomes table* below): did the baseline make something up, was your grounded answer right, and why.
6. Write down **one failure you still have**.

Linking on too little misses real connections. Linking on too much invents connections. The judges run a different, hidden pack that tests both, so fix your patterns, not individual questions.

Submit the notebook, run top to bottom on a fresh runtime (restart, then Run all), to **[submission method — add before sending]** by **[deadline — add before sending]**.

Two checks save time. `validate_registry()` tells you if a pattern is incomplete or its regex is broken. The graph cell prints how many links each pattern created: a pattern that links nothing adds nothing, and one that links a lot deserves suspicion.

## Filling in the outcomes table

After you run the public pack, the notebook asks for one line per question (P1-P8 and PP1-PP3), with three fields. Read the four lines the pack printed for each question: **Baseline** (the plain model with no graph), **Grounded** (your system), **Graph** (the verdict) and **Must say** (the right answer).

| Field | What it asks | How to decide |
|---|---|---|
| `baseline_fabricated` | Did the plain model **invent** something the records do not support? | Compare **Baseline** with **Must say**. `True` only if the baseline claimed a link, a cause or "same person" that the records do not show. If it said "no", said "cannot tell", or said "yes" to a link that really exists, write `False`. |
| `grounded_correct` | Was **your** grounded answer right? | The **Graph** line ends in `PASS` -> `True`; `FAIL` -> `False`. |
| `explanation` | Why did it come out that way? | One sentence about **your patterns**: which pattern matched (or did not), and whether the shared word was an ID or a place, level, amount or period. |

Three things to know:

- **Most baselines are right on most questions.** `False` is a normal, honest result. Writing `True` for something you did not see in the answer loses marks. You are scored on accurate, specific explanations, not on how many failures you found.
- **Explain with your patterns, not with what the text does not say.** Good: "*Both mention Room 214, but a room is a place, not an event ID, so no link.*" Or for a real link: "*Both records name Student S002 and our Student pattern extracts it, so they are connected.*" Weak: "*It is not specified*", "*Connected*", or "*it might have affected...*". Guessing is what the baseline does.
- **A real link is not a fabrication.** On the three "yes" questions (PP1-PP3) the link exists, so the baseline answering "yes" is correct and `baseline_fabricated` is `False`.

An example of the shape, not one of your questions:

```
{"id": "P0", "baseline_fabricated": False, "grounded_correct": True,
 "explanation": "Both mention Shelf 3, but a shelf is a place, not an event ID, so no link."},
```

## If something breaks

| Symptom | Likely cause | Fix |
|---|---|---|
| `ModuleNotFoundError` for `faiss` or `hnswlib` | The install cell did not finish or the runtime restarted | Run the `%pip install` cell again, then **Runtime → Restart session** and run all |
| The model download hangs for several minutes | Slow venue Wi-Fi | Wait; it is a one-time download. If it still stalls after five minutes, tell a mentor |
| `Secret not found` or an empty API key | The secret name is misspelled or notebook access is off | Check the name matches exactly, in capitals (`LLM_API_KEY`), and that the notebook access toggle is on |
| The LLM cell fails with an authentication or model-not-found error | Wrong key, or `LLM_MODEL` / `LLM_BASE_URL` does not match your provider | Check both against your provider's documentation |
| Rate-limit or quota errors from the LLM | Free-tier limits | Wait a minute and re-run the cell |
| `ServiceUnavailable` from Neo4j | The Aura instance paused, or an old connection went stale | Open the Aura Instances page, resume it, then re-run the connection cell |

Still stuck? Message **[contact or channel — add before sending]** with a screenshot of the error. Mentors will be walking the room during the lab specifically to fix setup problems, so ask early.